2 — Performance evaluation

Machine Learning (MPA-MLR)

Tomáš Vičar
Department of Biomedical Engineering · FEEC, Brno University of Technology

2026

Outline

1 · Supervised pipeline

From a question to a tested model.

2 · Performance evaluation definition

What exactly are we measuring, and on which data?

3 · Evaluation metrics

Accuracy, F_1, ROC, the threshold, calibration, R^2, multiclass averaging.

4 · Class imbalance

When the majority class hides everything.

5 · Validation approaches

Hold-out and k-fold cross-validation.

6 · Preprocessing

Scaling, imputation, encoding — without leakage.

1 · Supervised pipeline

> Before the first metric there is one question: what does the road from raw data to a tested model actually look like?

Supervised Learning Pipeline

1 · Define goal

What do we want to predict?

2 · Split data

Training / validation / test.

3 · Preprocess

Clean and prepare features.

4 · Train & tune

Fit models, adjust settings.

5 · Evaluate

Check performance on validation.

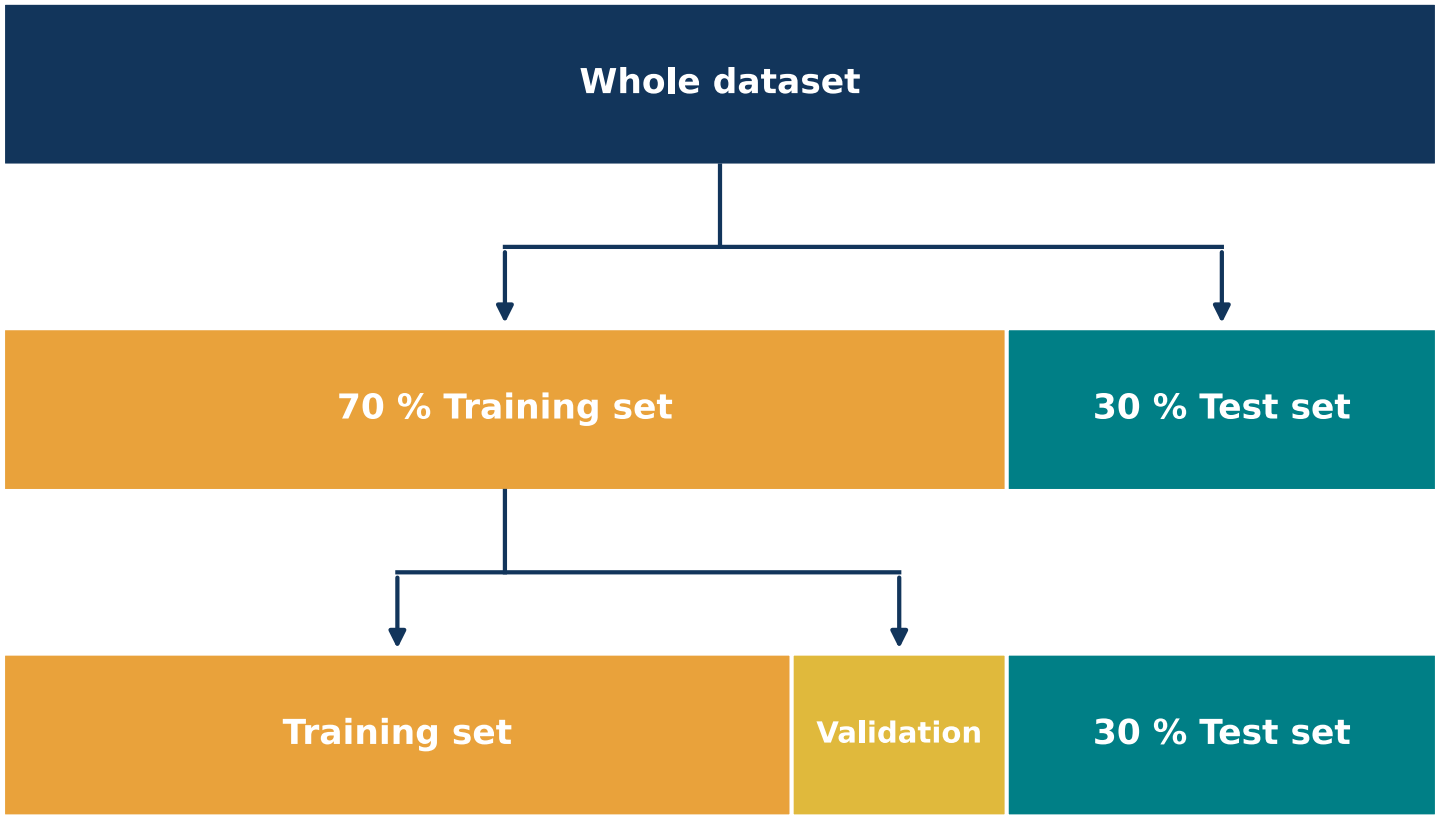
6 · Final test

Use test set once for a real estimate.

2 · Performance evaluation definition

> Training error can be pushed almost to zero. What we care about is the number the model gets on data it has never seen.

Dataset division



- **Training set** – for model training
- **Validation set** – used to tune hyperparameters, compare models, and prevent overfitting
- **Test set** – kept aside until the end, provides an unbiased estimate of generalization performance

Supervised Learning Goal

Ultimate goal: Not to minimize training error, but to find a model that performs well on *unseen data*.

Population risk (ideal but unknown):

$$R(\theta) = \mathbb{E}_{(\mathbf{x},y)\sim P}\left[\mathcal{L}(f_\theta(\mathbf{x}), y)\right]$$

P = true (unknown) data distribution; $R(\theta)$ = expected loss on future samples; $\mathcal{L}$ = some evaluation metric.

Practical approximation:

- We train by minimizing the *empirical risk* on training data

$$\hat{R}_{\text{train}}(\theta) = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \mathcal{L}(f_\theta(\mathbf{x}_i), y_i)$$

- But what we care about is the expected risk $R(\theta)$.
- Therefore we evaluate on a separate **test set**

$$\hat{R}_{\text{test}}(\theta) = \frac{1}{n_{\text{test}}} \sum_{j=1}^{n_{\text{test}}} \mathcal{L}(f_\theta(\mathbf{x}_j), y_j) \;\approx\; R(\theta).$$

Whole model development = optimization of generalization.

Evaluating on a Test Set

Key idea: The test set simulates *new, unseen samples* that the classifier will face in practice.

- During training, the model sees only the **training set**.
- If we also measure performance on the training set, we may get an **overly optimistic** result.
- By setting aside a **test set** (not used in training), we get a realistic estimate of how well the model generalizes.
- The test set should be drawn from the same distribution as future data.
- Test performance $\approx$ expected performance on future unseen samples.

Why evaluate performance?

- Get a reliable estimate of the classifier performance — i.e., how it will perform on new, so far unseen, data instances.
- Compare the classifiers you have developed — to decide which one is “the best”.

Model Development as Optimization Problem

Ultimate goal: Find the *whole pipeline* (features, preprocessing, model, hyperparameters) that minimizes prediction error on unseen data.

Abstract formulation:

$$\mathcal{P}^\star = \arg\min_{\mathcal{P}\in\mathcal{H}} \hat{R}_{\text{val}}(\mathcal{P})$$

- $\mathcal{P}$ = complete pipeline (feature extraction, preprocessing, model, hyperparameters)
- $\mathcal{H}$ = set of all candidate pipelines
- $\hat{R}_{\text{val}}(\mathcal{P})$ = estimated risk on a held-out validation set (the test set is kept for the final estimate)

Whole model development = pipeline optimization for the lowest error on unseen data.

Two Optimization Problems in Supervised Learning

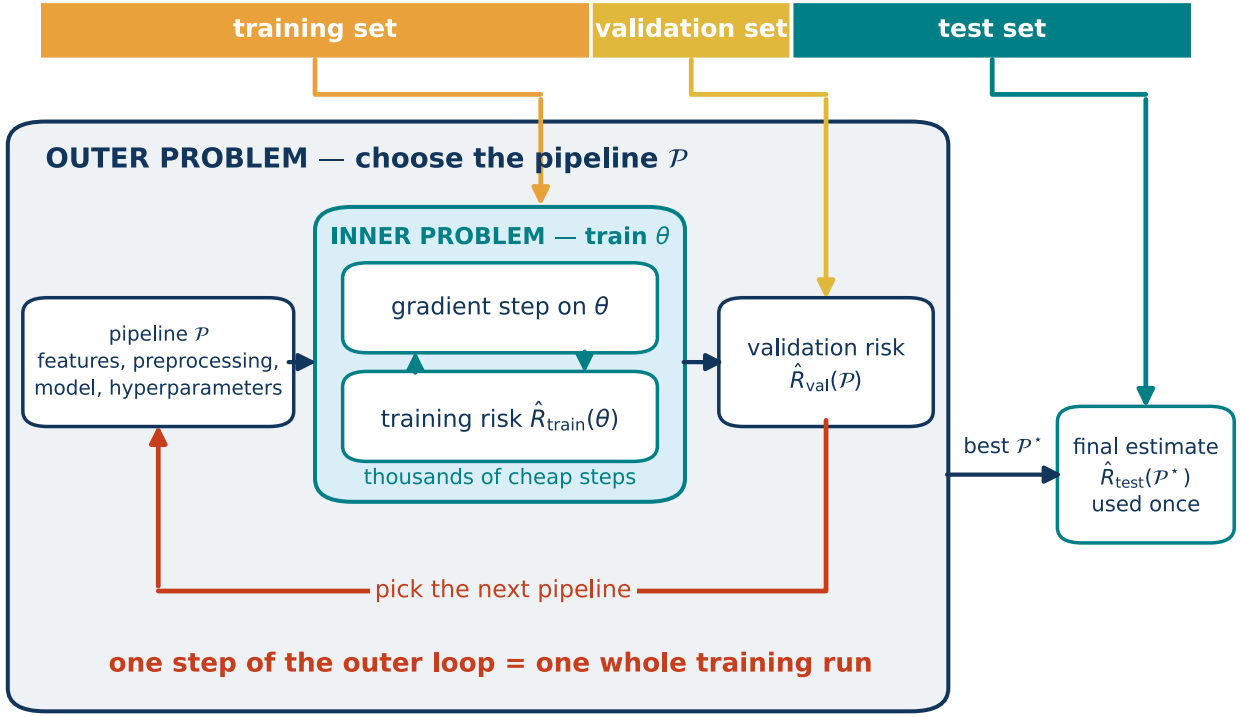
Inner problem — training (SGD, backpropagation, …): the parameters θ fit the **training set**

$$\theta^\star = \arg\min_\theta \hat{R}_{\text{train}}(\theta)$$

Outer problem — model development: the pipeline $\mathcal{P}$ is scored on the **validation set**

$$\mathcal{P}^\star = \arg\min_{\mathcal{P}\in\mathcal{H}} \hat{R}_{\text{val}}(\mathcal{P})$$

Test set: $\hat{R}_{\text{test}}(\mathcal{P}^\star)$ — once, at the very end.



Role of the Validation Set

Problem: If we tune hyperparameters or select features using the test set, we “cheat” and performance will be too optimistic.

Solution: Introduce a **validation set**:

- Separate from training data.
- Used to tune hyperparameters, select features, compare models.
- Prevents **overfitting to the test set**.

Workflow:

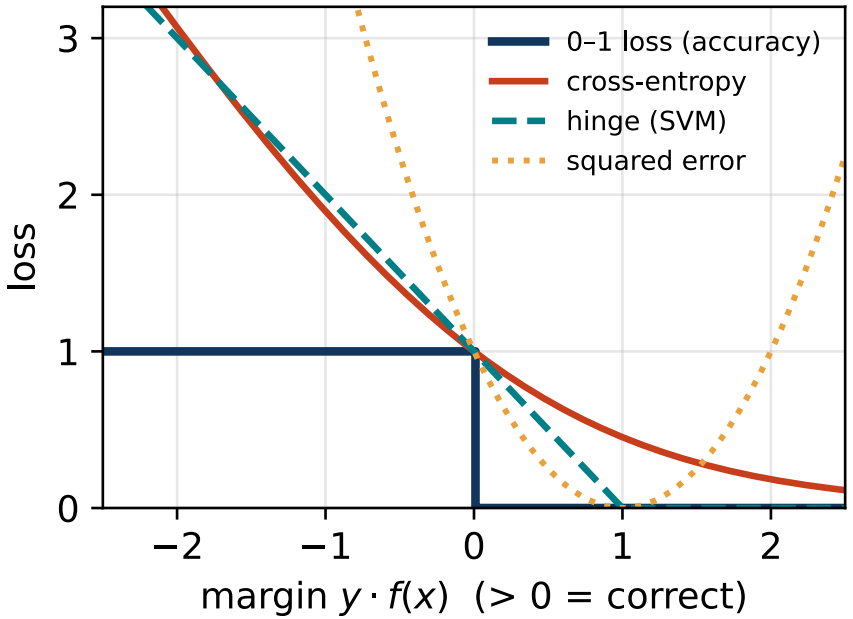
- Train $\rightarrow$ Validate $\rightarrow$ Tune $\rightarrow$ (repeat)
- After the final model is chosen, use the **test set only once** for unbiased evaluation.

We Need a Measure of Performance

Why? Optimization means “searching for the best” — first define what *best* means: a number that says how bad a prediction is.

	loss (training)	**metric (evaluation)**
used by	the optimizer, every step	you, on validation / test data
must be	differentiable, per sample	what the task cares about
examples	cross-entropy, MSE, hinge	accuracy, F_1, AUC, Dice, MAE

Why they differ: accuracy is a **step** in the model’s output (zero gradient almost everywhere); F_1 and AUC are computed on the **whole set**, not per sample. So we train on a smooth **surrogate** and report the metric. Sometimes they coincide (MSE in regression).



Without a loss function, the computer would not know which model is better.

3 · Evaluation metrics

> A single number never describes a model completely. Let’s see what each number measures — and what it hides.

Basic Evaluation Metrics

Classification — accuracy (and error rate $= 1 - \text{Accuracy}$)

$$\text{Accuracy} = \frac{\#\{\hat{y}_i = y_i\}}{n} = \frac{TP + TN}{TP + TN + FP + FN}$$

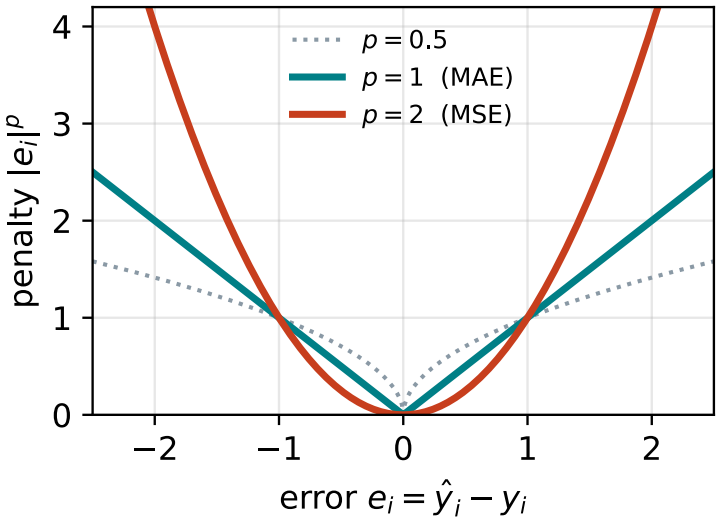
Regression — MSE and MAE are means of the error $e_i = \hat{y}_i - y_i$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n} e_i^2 \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |e_i|$$

The same thing as norms of the error vector $\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y}$:

$$\text{MAE} = \frac{1}{n}\|\mathbf{e}\|_1 \qquad \text{MSE} = \frac{1}{n}\|\mathbf{e}\|_2^2 \qquad \text{RMSE} = \sqrt{\text{MSE}} = \frac{1}{\sqrt{n}}\|\mathbf{e}\|_2$$

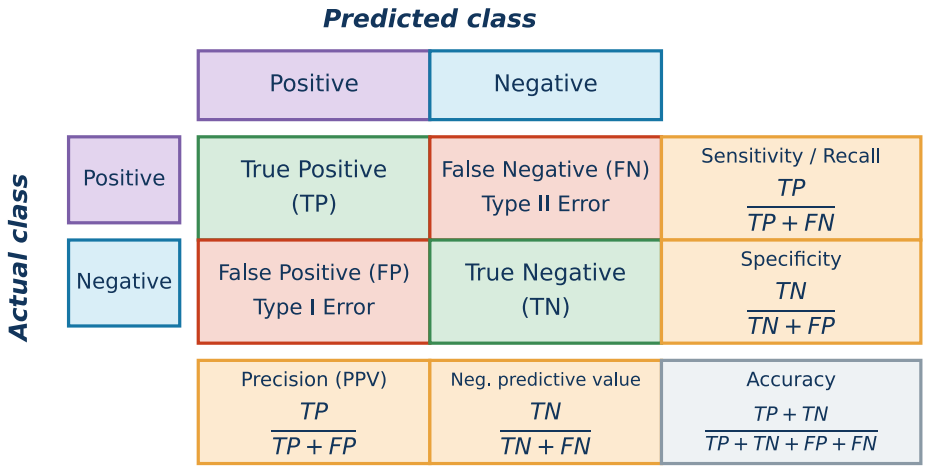
in general $\frac{1}{n}\|\mathbf{e}\|_p^p$ with $\|\mathbf{e}\|_p = \left(\sum_i |e_i|^p\right)^{1/p}$ — p sets the cost of a **large** error.



By hand: $y = [1, 0, 1, 2, 0], \hat{y} = [1, 1, 1, 0, 0] \Rightarrow$ accuracy $= 3/5 = 0.6$. Regression: $y = [3, -0.5, 2, 7, 4.2]$, $\hat{y} = [2.5, 0, 2.1, 7.8, 3.5] \Rightarrow \mathbf{e} = [-0.5, 0.5, 0.1, 0.8, -0.7]$, MSE $= 1.64/5 = 0.328$, MAE $= 2.6/5 = 0.52$, RMSE $= 0.573$.

code example

Confusion Matrix (Binary) based metrics



Sensitivity, specificity, PPV and NPV are not sufficient alone — a model that predicts only the positive class has sensitivity 1.

$$F_1\ (\text{Dice}) \;=\; 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \;=\; \frac{2TP}{2TP + FP + FN} = \frac{2|X \cap Y|}{|X| + |Y|}$$

Harmonic mean of precision and recall. Useful in cases with a small fraction of positive samples.

By hand: 10 samples, $TP = 4, FP = 2, FN = 1, TN = 3 \Rightarrow$ precision $4/6 = 0.67$, recall $4/5 = 0.80$, $F_1 = 8/11 = 0.73$; the always-positive model: recall 1.0, $F_1 = 10/15 = 0.67$.

code example

ROC: Intuition

Setting: A classifier outputs a *score* (confidence); we pick a *threshold* to decide Positive/Negative. Sometimes the threshold is part of the model (e.g. SVM).

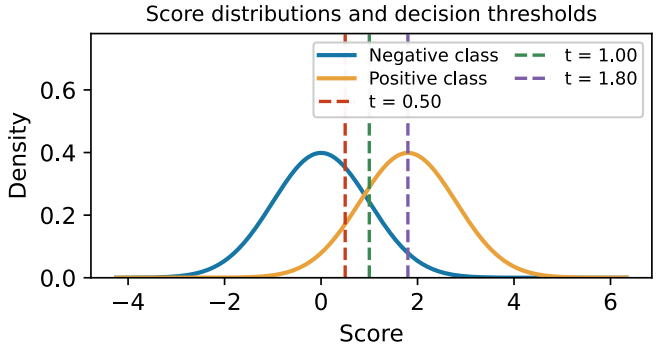
What the score is: logistic regression / neural net — the sigmoid or softmax output; **random forest** — the fraction of trees voting for the class; **kNN** — the fraction of the k neighbours; **SVM** — the signed distance from the hyperplane; **naive Bayes** — the posterior.

ROC curve shows the trade-off

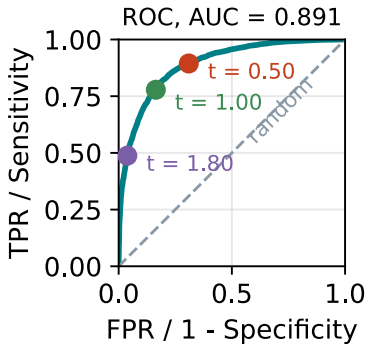
$$\text{TPR (Recall, Sensitivity)} = \frac{TP}{TP + FN}$$

$$\text{FPR (1 - Specificity)} = \frac{FP}{FP + TN}$$

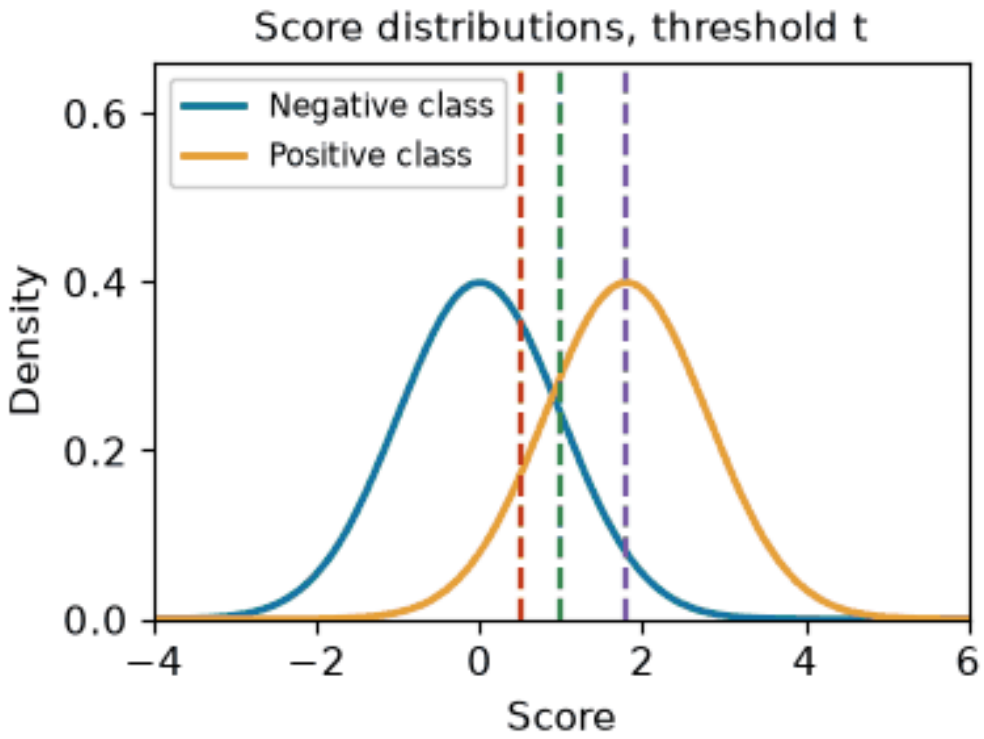
as we sweep the threshold.

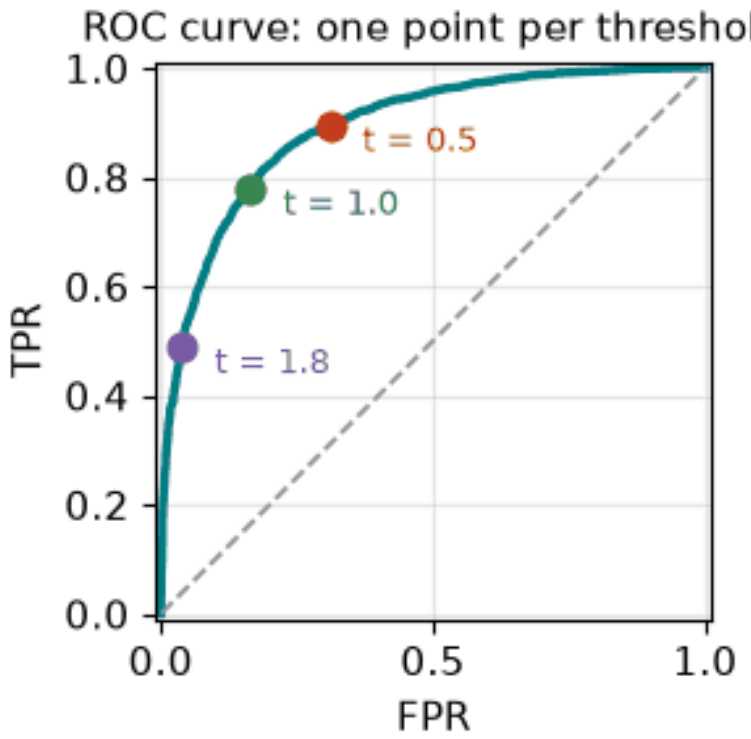


- Each threshold $\Rightarrow$ one point (FPR, TPR).
- The **ROC curve** connects these points.
- **Perfect** classifier touches $(0, 1)$.
- **Random** classifier lies on the diagonal.
- More information than just one value.



ROC: Sweeping the Threshold





- Shaded areas = the fraction of each class **above** the threshold: orange = TPR, blue = FPR.
- Threshold high $\Rightarrow$ almost nothing is called positive: $(0, 0)$. Threshold low $\Rightarrow$ everything is positive: $(1, 1)$.
- The whole curve is one classifier; choosing the threshold only picks a point on it.

Constructing the ROC Curve and AUC

Procedure

1. Compute scores s_i for all samples.
2. Sort unique score values and use them as thresholds.
3. For each threshold, binarize predictions and compute TPR, FPR.
4. Plot $(\mathrm{FPR}, \mathrm{TPR})$ and connect.

ROC AUC (area under curve)

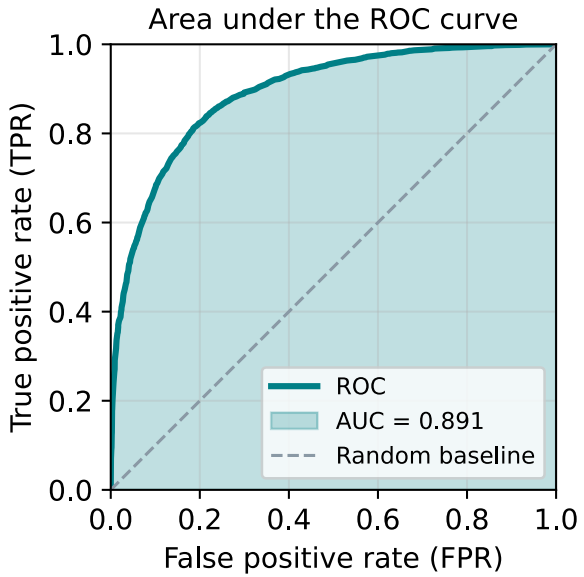
$$\mathrm{AUC} = \int_0^1 \mathrm{TPR}(\mathrm{FPR})\, d(\mathrm{FPR})$$

Interpretation:

$$\mathrm{AUC} = \Pr\left(s(\mathbf{x}^+) > s(\mathbf{x}^-)\right)$$

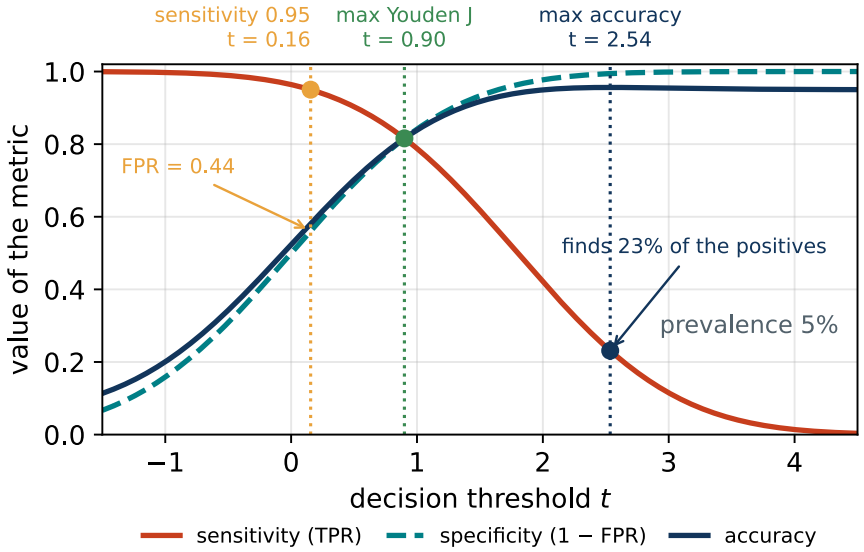
AUC = probability that a randomly chosen positive has a higher score than a randomly chosen negative.

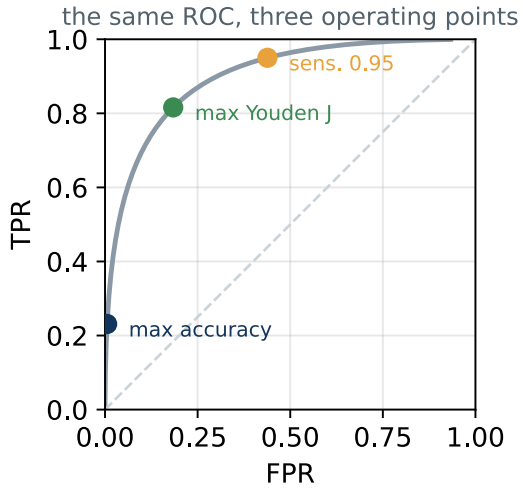
code example



Choosing the Operating Point

The ROC is the whole classifier; the **threshold** picks one point on it — the default 0.5 on the predicted probability is a habit, not a rule. Same scores as before, prevalence 5 %:

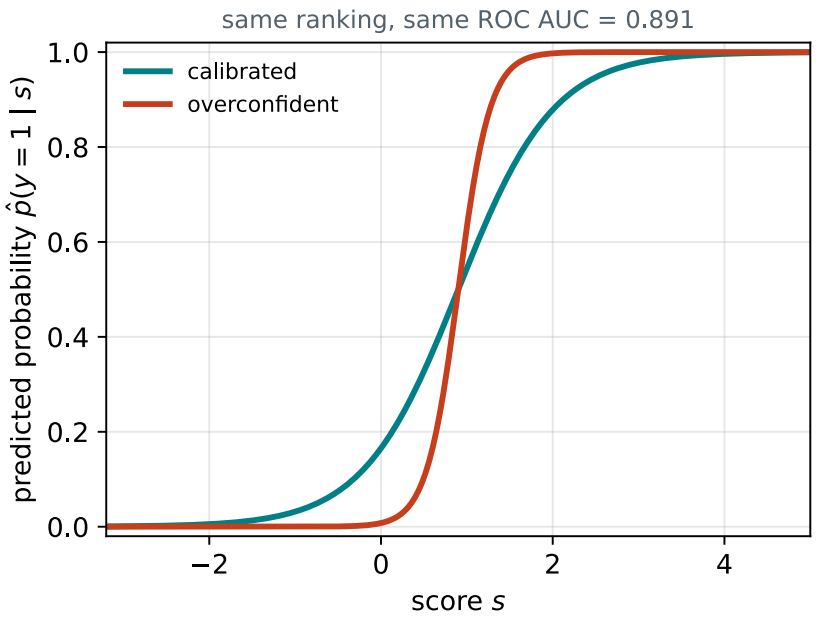


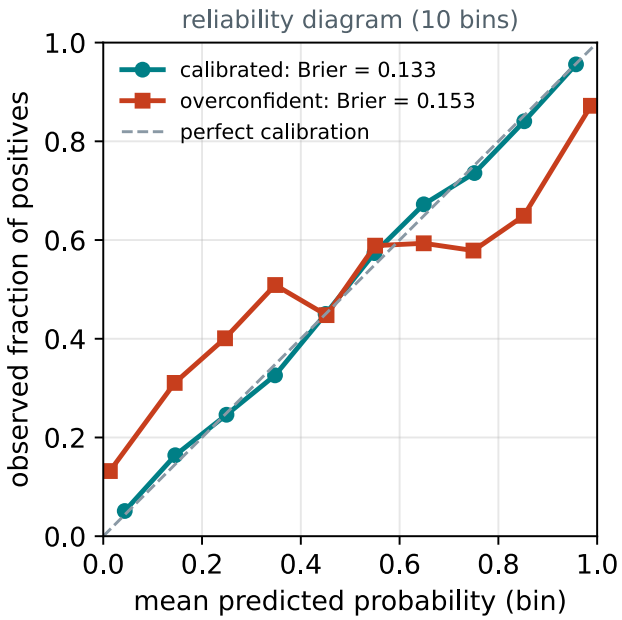


- **Max accuracy** (equal cost of FP and FN): $t = 2.54$, accuracy 0.956 — but it finds only **23 %** of the positives.
- **Youden's** $J = \mathrm{TPR} - \mathrm{FPR}$ (the point farthest from the diagonal): $t = 0.90$, sensitivity $=$ specificity $= 0.82$.
- **Required sensitivity** (screening: miss at most 5 %): $t = 0.16$, paid for by FPR $= 0.44$.

code example

Calibration: Is the Score a Probability?





- **Calibrated:** among the samples with $\hat{p} = 0.8$, about 80 % are positive. The **reliability diagram** checks this bin by bin.
- A **monotone** transformation of the score keeps the ranking, the ROC and the AUC — and can ruin the probabilities. **AUC does not see calibration.**
- The **Brier score** $= \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i)^2$ — the MSE of the probabilities — does: 0.133 vs 0.153.

code example

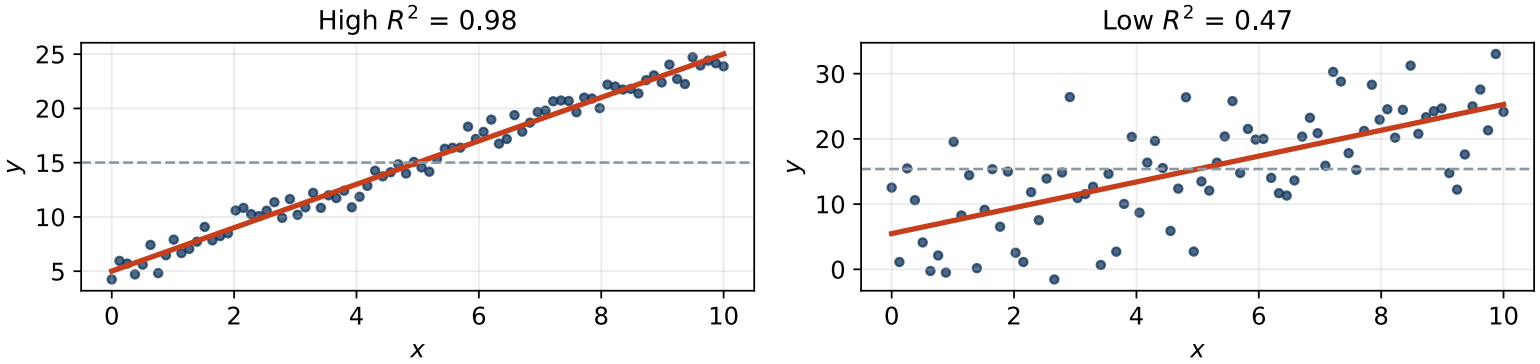
R^2 (Coefficient of Determination)

Idea: How much better the model is than always predicting the mean.

Definition

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

- Numerator = sum of squares error (residual), SSE
- Denominator = sum of squares total, SST
- $R^2 \in (-\infty, 1]$; closer to 1 means a better fit. Prediction of the mean $\rightarrow 0$.



code example

Multiclass vs. Multilabel

Multiclass: exactly one label out of K classes (single-label) — only one can be true

$$y \in \{1, \dots, K\} \quad \text{or one-hot} \quad \mathbf{y} \in \{0,1\}^K \text{ with } \sum_{k=1}^{K} y_k = 1$$

(one-hot example: $\mathbf{y} = [0, 1, 0, 0, 0]$)

Examples:

- Brain tumor type on MRI: $\{\text{Meningioma}, \text{Glioma}, \text{Pituitary}\}$
- Diabetic retinopathy severity: $\{\text{No}, \text{Mild}, \text{Moderate}, \text{Severe}, \text{PDR}\}$

Multilabel: zero, one, or many labels per sample (multi-hot) — several can be true

$$\mathbf{y} \in \{0,1\}^K \quad (\text{e.g., } \mathbf{y} = [1, 0, 0, 1, 1])$$

Examples:

- Image tags: {*cat*, *outdoor*, *night*}
- Patient comorbidities: multiple diseases can be present simultaneously

Basic Metrics: Multiclass vs. Multilabel

We can aggregate metrics for individual classes/labels.

Averaging schemes (for multi-class / multi-label)

- **Micro:** pool all decisions across classes/labels first (TP, FN, …), then compute one global value (reflects frequent classes more).
- **Macro:** compute the per-class/label metric, then average (treats all classes equally).
- **Weighted macro** uses class supports (counts).

Multiclass (single label)

- **Accuracy** — overall fraction correct
- **Macro-F_1** — average F_1 across classes (handles imbalance)

Multilabel (multi-hot)

- **Subset accuracy (exact match)** — all labels must be correct
- **Hamming loss** — per-label error rate (lower is better)
- **Micro/Macro F_1** — aggregate across labels

Macro vs Weighted vs Micro

Actual \ Predicted	Airplane	Boat	Car
Airplane	2	1	0
Boat	0	1	0
Car	1	2	3

Label	TP	FP	FN	Prec.	Rec.	F1
Airplane	2	1	1	0.67	0.67	0.67
Boat	1	3	0	0.25	1.00	0.40
Car	3	0	3	1.00	0.50	0.67
Total	**6**	**4**	**4**			

support = 3 : 1 : 6 (of 10 samples)

Macro-F1

$$\frac{0.67 + 0.40 + 0.67}{3} = 0.58$$

Weighted-F1

$$0.67 \cdot 0.3 + 0.40 \cdot 0.1 + 0.67 \cdot 0.6 = 0.64$$

Micro-F1

$$\frac{TP}{TP + \frac{1}{2}(FP + FN)} = \frac{6}{6 + \frac{1}{2}(4 + 4)} = 0.60$$

Macro-average — treat all classes equally, so rare classes matter. Use it under class imbalance when you want fairness across classes. Highlights poor performance on minority classes, but can be noisy for tiny classes.

Weighted macro-average — like macro, but weighted by class support (prevalence). Use it when you want class fairness *and* to reflect real-world frequencies. More stable than plain macro when some classes are very small — a good default under imbalance.

Micro-average — pools all decisions, measuring overall instance-level performance. Use it when overall volume matters; it may be dominated by majority classes. In single-label multiclass, micro-F_1 equals accuracy.

code example

4 · Class imbalance

> When only one percent of the samples are positive, a model that never finds anything is 99 % accurate.

Unbalanced Datasets (Class Imbalance)

- One class is much rarer than others (e.g., positives $< 5\%$).
- **Accuracy paradox:** high accuracy even if the minority is ignored.

Evaluate properly

- Use **macro** / **weighted F_1**.
- Show a **normalized confusion matrix** and report prevalence.

Training options

- **Class weights** / **cost-sensitive** loss.
- **Resampling:** undersample the majority, oversample the minority.

n = 1000, prevalence 5% (50 positives)

Always "negative"

Actual \ Predicted	Pos.	Neg.
Pos.	0	50
Neg.	0	950

Accuracy	**0.95**
Recall	**0.00**
F_1 (positive)	**0.00**
Macro-F_1	**0.49**

Real classifier (t = 1)

Actual \ Predicted	Pos.	Neg.
Pos.	39	11
Neg.	151	799

Accuracy	**0.84**
Recall	**0.78**
F_1 (positive)	**0.33**
Macro-F_1	**0.62**

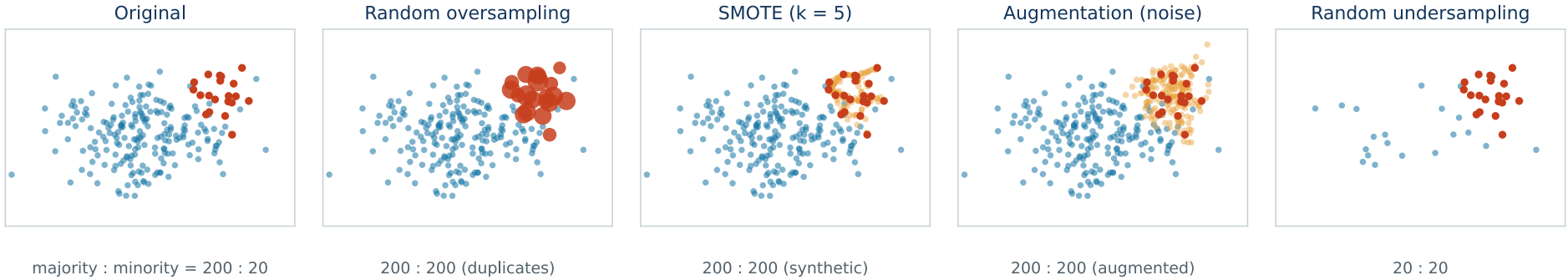
Handling Imbalance: Oversampling vs. Undersampling

Oversampling (increase minority)

- *Random oversampling*: duplicate minority examples; most popular.
- *SMOTE, ADASYN*: synthesize new minority points between neighbours.
- **Augmentation**: label-preserving transforms (flip, rotate, noise) — the SMOTE of images and signals.
- **Pros**: keeps all majority data; boosts recall. **Cons**: overfitting (duplicates), artifacts (synthetic). **Use when**: very small minority.

Undersampling (reduce majority)

- *Random undersampling*: drop majority examples; most popular.
- Cleaning rules: Tomek links, ENN, NearMiss, ClusterCentroids.
- **Pros**: faster; can remove borderline noise/overlap. **Cons**: discards information; possible underfitting.
- **Use when**: huge majority class; training cost is high.



Best Practices (Sampling & Splits)

Pipeline & leakage

- Apply any **sampling only on the training data**, **inside each CV fold**; never on validation/test.
- Do preprocessing (scaling, PCA) *inside the split* too, fitted on training only. Values from the training data are then used for validation/test.

Stratify splits (important)

- Keep class proportions similar in train/val/test and in each fold — stratified.
- With groups (patients/devices/sites): put all data from each group into a single split → groups don't leak across splits.

  E.g., when multiple signals come from the same patient, assign all of that patient's signals to a single split. Otherwise you get leakage and an overoptimistic result, because signals from the same patient are highly correlated.

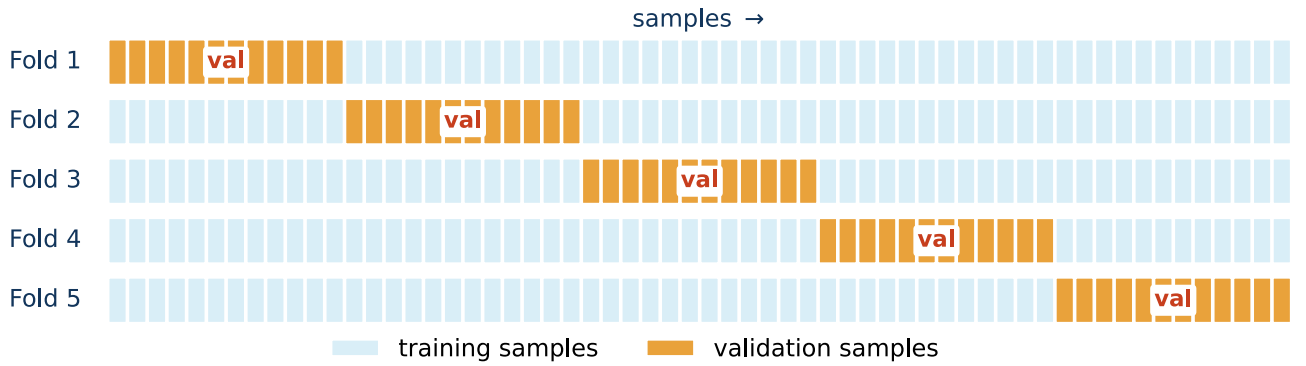
5 · Validation approaches

> One split of the data gives one number. How much of that number is the model, and how much is the luck of the split?

Cross-Validation (k-fold)

- Split data into k (roughly) equal folds; repeat k rounds.
- Each round: **train** on $k-1$ folds, **validate** on the remaining fold.
- Report the mean $\pm$ std of the validation metric across folds.
- Use **stratified** (and **group**-aware) CV when classes or groups exist.
- Keep the **test set untouched**; scaling/sampling happen *inside each fold*.
- If $k = n$ (number of samples), it is **leave-one-out** CV.
- With small datasets it can be used for testing as well.

$$\uparrow k \Rightarrow \uparrow \text{computation (more fits)} \qquad \downarrow k \Rightarrow \downarrow \text{training data per fit } \left(n\frac{k-1}{k}\right)$$



code example

Hold-out Validation (Single Split)

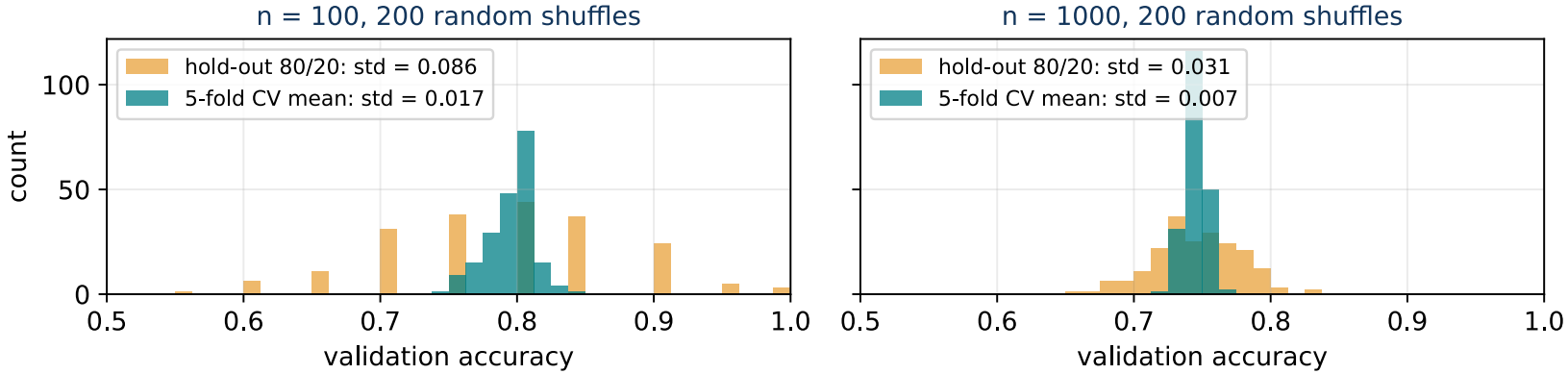
- Split data *once* into **train** / **validation** / **test** (e.g. 70/15/15 or 60/20/20).
- Train on **train**; tune hyperparameters/thresholds on **validation**; report the final performance on **test (once)**.

Pros

- Simple and fast (only one fit per model/setting).
- Works well when the dataset is large.
- Easy to keep a clean, untouched test set.

Cons

- Higher variance estimate — results depend on the particular split.
- With limited data, you may waste samples that could help training.



code example

6 · Preprocessing

> The model only ever sees preprocessed data. That is exactly where most of the leakage is created.

Preprocessing

Why it matters

- Models assume comparable feature scales, no NaNs, and sensible encodings.
- **Preprocessing must be fitted *only* on training data** and applied to val/test.

Typical steps

- **Cleaning:** missing values, outliers, inconsistent categories.
- **Scaling:** standardization / normalization / robust scaling.
- **Encoding:** one-hot, ordinal, (careful) target encoding.
- **Dimensionality reduction:** PCA, feature selection.

Implement preprocessing inside a `Pipeline` and evaluate with CV.

Scaling & Normalization

Standardization (z-score) — mean 0, std 1; keeps the shape of the distribution; the default for linear models, SVM, PCA, neural nets

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Min–max — exactly $[0, 1]$; for bounded inputs (images, sigmoids); one outlier squeezes everything else

$$x' = \frac{x - \min_{\text{train}}}{\max_{\text{train}} - \min_{\text{train}}}$$

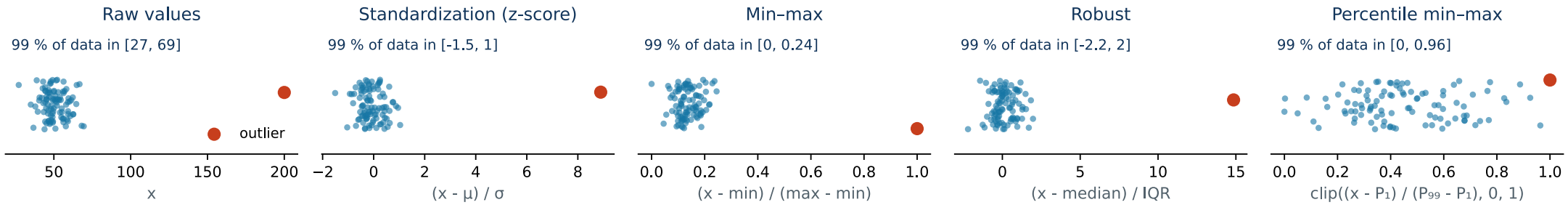
Robust — median and IQR instead of μ and σ; outliers do not move the scale

$$x' = \frac{x - \text{median}_{\text{train}}}{\text{IQR}_{\text{train}}}$$

Percentile min–max — bounded *and* robust: the extremes are clipped to the 1st and 99th percentile

$$x' = \text{clip}\left(\frac{x - P_{1,\text{train}}}{P_{99,\text{train}} - P_{1,\text{train}}},\ 0,\ 1\right)$$

Tips: trees usually *don't* need scaling; linear / SVM / kNN / logistic regression *do*. **Fit** on train folds only, **transform** val/test.



code example

Missing Values, Outliers & Encodings

Imputation (replacement of missing values)

- Simple: mean / median / most-frequent; add a *missingness indicator*.
- Model-based: kNN imputer, iterative (MICE). Fit on the training fold only.

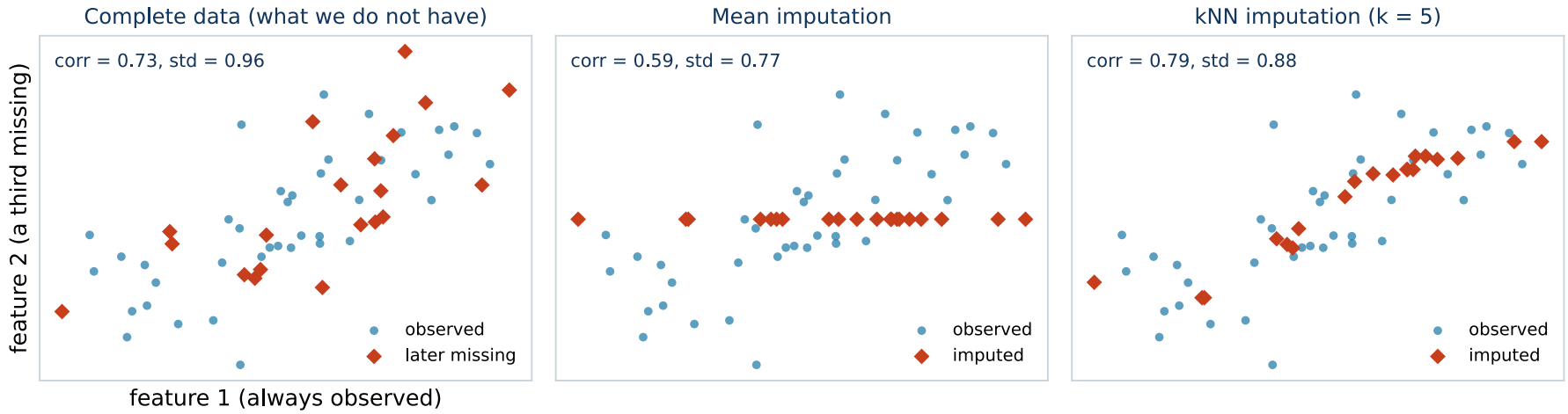
Outliers

- Prefer **robust scaling** or **winsorization** to blind removal.
- If removing, do it **inside** each CV training fold to avoid leakage.

Encodings

- **One-hot** for nominal features (watch out for high cardinality).
- **Ordinal** only if the order is meaningful.
- **Target/mean encoding**: powerful, but **must** be done with CV-inside-train (out-of-fold averages) to prevent leakage.

Imputation: Filling the Missing Values



- **Mean / median / most frequent** — one number per column; **flattens** it (std $0.96 \to 0.77$) and weakens its relations (corr $0.73 \to 0.59$). Median survives outliers.
- **Missingness indicator** — a 0/1 column "was missing": often the most informative feature (*not examined* $=$ the doctor saw no need).
- **kNN imputer** — mean of the k nearest rows on the other features; **iterative (MICE)** — each column regressed on the others. Both follow the trend (corr 0.79), a bit too smoothly.
- **Dropping rows** — only when few and missing at random (lab 3: a missing `gender`).

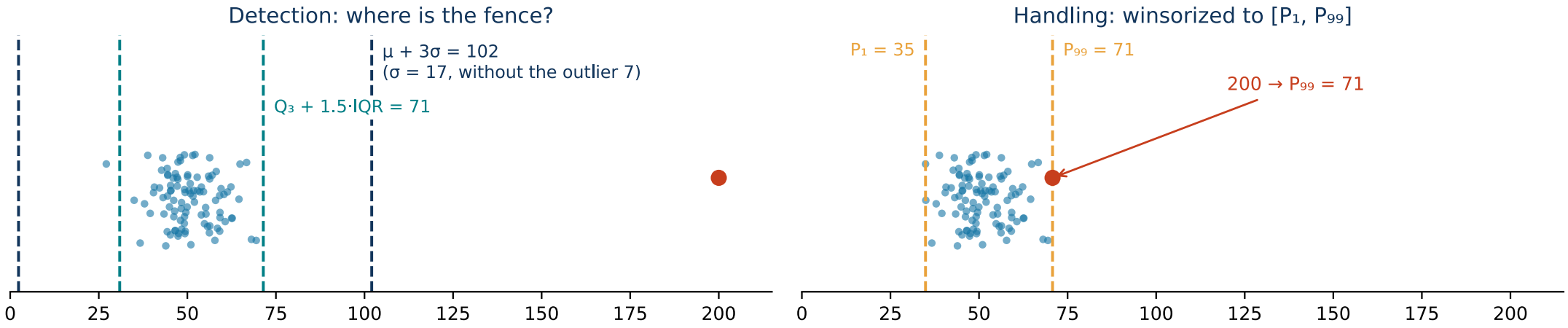
Imputation: a Worked Table

the raw table (— = missing)			*imputed (fitted on the training rows)*					*indicator columns*	
age	**height (cm)**	**score**	**age mean**	**height mean**	**height median**	**score mean**	**score → 0**	**height missing?**	**score missing?**
20	172	68	20	172	172	68	68	0	0
21	—	74	21	**210.0**	**173.5**	74	74	1	0
19	165	—	19	165	165	**71.7**	**0**	0	1
23	180	91	23	180	180	91	91	0	0
—	400	55	**20.9**	400	400	55	55	0	0
20	168	62	20	168	168	62	62	0	0
22	—	80	22	**210.0**	**173.5**	80	80	1	0
21	175	—	21	175	175	**71.7**	**0**	0	1

the 400 cm row drags the mean to 210 "0" = a decision, not a value

- **Mean** height 210.0: one of six observed values is 400, so the mean copies the outlier into two more rows. **Median** 173.5 ignores it.
- **Score** $\rightarrow 0$ says “did not take the test”; the **mean** 71.7 says “an average student”. Neither is neutral — the **indicator** column keeps the fact.
- All statistics come from the **training rows**; validation and test rows get the same numbers.

Outliers: Detection and Handling



- **Detection:** z-score $|x - \mu| > 3\sigma$ — but σ itself is inflated by the outlier (17 instead of 7). **Tukey fences** $Q_1 - 1.5\,\mathrm{IQR}$, $Q_3 + 1.5\,\mathrm{IQR}$ are robust. Multivariate: isolation forest, local outlier factor.
- **Keep** it and use **robust scaling** — the safest default; an outlier need not be an error, it may be *the* interesting patient.
- **Winsorize** — clip to $[P_1, P_{99}]$: the value stays, its leverage goes ($200 \to 71$).
- **Remove** only what is impossible (a height of 400 cm) — inside the training fold, with the rule fitted there.

Outliers: a Worked Table

height (cm)	z-score $(x - \mu)/\sigma$	$\lvert z \rvert > 3$?	outside Tukey fence?	winsorized to the fence	decision
172	-0.37	no	no	172	keep
165	-0.46	no	no	165	keep
180	-0.26	no	no	180	keep
400	+2.64	**no**	**yes**	**193.5**	**remove (impossible)**
168	-0.42	no	no	168	keep
175	-0.33	no	no	175	keep
161	-0.51	no	no	161	keep
177	-0.30	no	no	177	keep

$\mu = 199.8$, $\sigma = 75.9$ (without the 400: 171.1, 6.3) — one value inflates σ twelvefold, so its $z = 2.6 < 3$: the rule misses its own cause

$Q_1 = 167.2$, $Q_3 = 177.8$, IQR = 10.5 — fence $Q_3 + 1.5 \cdot \text{IQR} = 193.5$: the quartiles do not move, the fence catches it

- **z-score rule misses it:** the 400 inflates σ from 6.3 to 75.9, so its own z is only 2.6 — *masking*; in eight values no $\lvert z \rvert$ can reach 3.
- **Tukey fence catches it:** quartiles ignore the extreme, $Q_3 + 1.5\,\text{IQR} = 193.5$.
- **Winsorize** to the fence ($400 \to 193.5$) if the value is plausible; **remove** if impossible — 400 cm is.

Encoding Categorical Features

		one-hot				*ordinal — wrong for nominal*	*target encoding = mean of y per category*	
city	**y**	**Brno**	**Prague**	**Ostrava**	**Plzeň**	**ordinal code**	**target (in-sample)**	**target (smoothed, m = 2)**
Brno	1	1	0	0	0	0	0.67	0.62
Brno	1	1	0	0	0	0	0.67	0.62
Brno	0	1	0	0	0	0	0.67	0.62
Prague	1	0	1	0	0	1	0.50	0.53
Prague	0	0	1	0	0	1	0.50	0.53
Ostrava	0	0	0	1	0	2	0.33	0.42
Ostrava	1	0	0	1	0	2	0.33	0.42
Ostrava	0	0	0	1	0	2	0.33	0.42
Plzeň	1	0	0	0	1	3	**1.00**	0.70

one row → the encoding is its own label
(leak: the feature knows the answer)

global mean = 0.56

- **One-hot** — one 0/1 column per level; they sum to 1, so drop one for linear models. Many levels: group the rare ones into *other*.
- **Ordinal code** — one integer; only when the order is real ($\textit{low} < \textit{medium} < \textit{high}$). For a nominal feature it invents an order the model will use.
- **Target encoding** — the mean of y per level, one column for thousands of levels. In-sample it **leaks**: a one-row level is encoded by its own label. Fix: **out-of-fold** means on the training data, **smoothed** towards the global mean.

Recap Quiz I — Concepts & Splits

1. What is the difference between **empirical risk** and **expected (population) risk**, and which one do we ultimately care about?
2. Why must the **test set** be used exactly once? What goes wrong if you “peek”?
3. Explain the roles of **train**, **validation** and **test** in one sentence each.
4. Give two examples of **data leakage**. How would you prevent each one?
5. When should you use **stratified** vs. **group**-aware splitting?
6. In k-fold CV, what happens (pros/cons) when you **increase** k?
7. Which models **need scaling** and which typically don’t? Give examples.

Recap Quiz II — Metrics & Imbalance

1. When is **accuracy** misleading? Name two better metrics for rare positives.
2. Why is the threshold 0.5 not a rule? Name two ways of choosing the **operating point** and say which data you choose it on.
3. How do you interpret **ROC AUC**? Why can a model with a high AUC still give useless **probabilities**, and which plot or score reveals it?
4. Define **precision**, **recall** and $\mathbf{F_1}$. What trade-off does the threshold control?
5. Give one **oversampling** and one **undersampling** technique. Where in CV do they belong?
6. For multiclass, explain **micro** vs. **macro** vs. **weighted** averaging.